Fitness Tracker

Silver Gjeka

University: University of Verona (UNIVR)
Subject: Machine Learning
Degree: Master's Degree in Artificial Intelligence
Matriculated: VR527645

Introduction

Fitness trackers are now on many people's wrists, helping them count steps, check heart beats, and track sleep. They've become needed tools in daily life because they help us understand our health better. With a quick look at your wrist, you can see if you have moved enough today or if your heart is working too hard. Trackers remind busy people to stand up and move when they've sat too long. They also help to show patterns over time, such as if you sleep better on days when you exercise. For people who are trying to be healthier, these devices provide the numbers and facts that keep them going. In this project, we will see how to track various gym exercises like bench press, squats, and more.

Machine Learning Role

Machine learning helps users automatically track their gym workouts by recognizing different exercises through motion data from sensors such as accelerometers and gyroscopes. Instead of manually logging exercises, users get real-time exercise detection. This makes it easier to stay consistent, monitor progress, and stay motivated throughout your fitness journey.

Dataset Description

The dataset consists of motion sensor data from 5 participants performing a variety of fitness exercises. The data includes accelerometer and gyroscope readings, with each set labeled according to the specific exercise. The data set covers six different exercises, including bench press, overhead press, squat, deadlift, row, and rest, and participants perform varying numbers of sets for each exercise.

The dataset includes the following variables:

- **Epoch (ms):** Millisecond timestamp of each sensor reading.
- **Accelerometer_x/y/z:** Measures body acceleration in three directions.
- **Gyroscope_x/y/z:** Measures rotational velocity in three directions.
- **Participant:** The person performing the exercise.
- **Label:** The specific exercise performed (e.g., squat).
- **Category:** Intensity level (e.g., light, medium, heavy).
- **Set:** Identifier for each repetition set.

Objective

The objective of this project is to develop a machine learning model capable of recognizing and classifying gym exercises in real time using sensor data from accelerometers and gyroscopes. To achieve this, we will experiment with various models like Support Vector Machines (SVM), Random Forest and others, to identify which approach offers the best performance in predicting exercises. One challenge is making sure the model works well for different people, exercise types, and movement styles, so it can be used by anyone.

Studying the Dataset

In this project, we work with real-time movement data collected every 200 milliseconds (epoch) using wearable devices. These devices include two main sensors: an accelerometer and a gyroscope. The dataset includes data from multiple participants, each performing a variety of exercises from different categories. This variety helps train machine learning models to recognize different movement patterns across different people.

Accelerometers: measure how fast something is speeding up or slowing down in three directions: up/down,left/right, and forward/backward (X, Y, Z axes). This helps capture simple body movements like walking, jumping, or lifting weights. When you move your arm or leg, the accelerometer picks up that motion as changes in speed.

Gyroscopes: measure how something is rotating or turning around the same three directions. They track rotation and balance, such as when you twist your body, bend forward, or turn to the side. This helps detect more complex movements that involve changes in body position.

Together, these sensors give us a full picture of what the body is doing both in terms of straight movement and rotation. By using the signals they produce, we can train a machine learning model to recognize which exercise a person is performing at any moment.

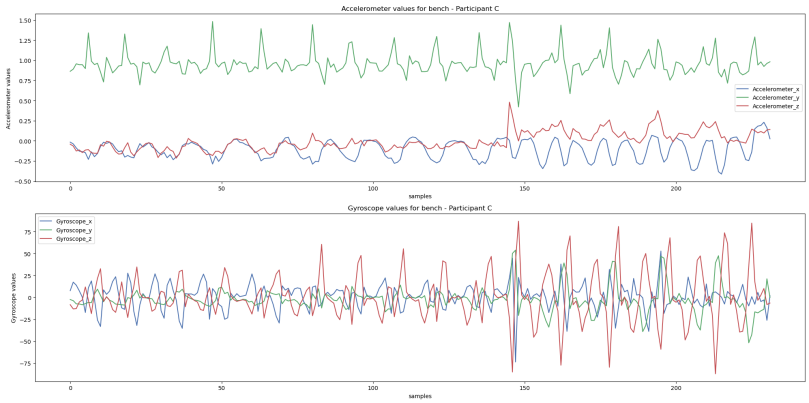

Figure 1: Accelerometer and Gyroscope Data from Participant.

Why This Dataset?

This dataset is a time series, it shows how movements change over time. This is great for tracking exercises. It has detailed data from accelerometers and gyroscopes, so we can see both straight and turning movements. The data is recorded very often, which makes it good for real-time predictions. It also helps learning how to work with signals. For example, trying things like smoothing the data or using sliding windows to get it ready for machine learning.

Data Preprocessing

Before using the sensor data, we need to clean it to make sure it's accurate and useful. One common problem is outliers unusual values caused by sensor errors, random noise, or sudden unexpected movements. By correcting these outliers, we avoid confusing the machine learning models and improve the accuracy.

Outliers

We analyze accelerometer and gyroscope data during exercises. Figures 2 show the boxplots.

Gyroscope: For example in a squat, the body moves mostly up and down, not much turning. So the gyroscope values stay close to zero. Even a small twist shows up as an outlier because it stands out from the rest of the data. This is why we see many outliers the normal values are tightly grouped, so small changes look big.

Accelerometer: Squats involve big up-and-down movements, which the accelerometer captures. The median line may not be in the center, showing that the motion isn't perfectly balanced. But since large changes are expected, we see fewer outliers.

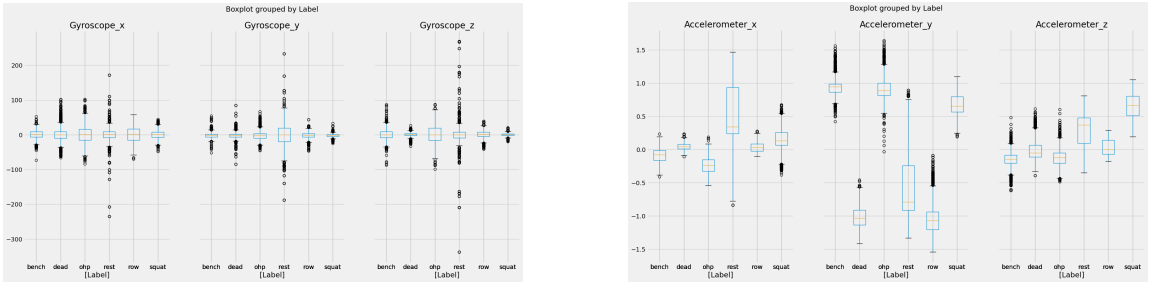

Figure 2: Outliers detected Gyroscope, Accelerometer.

Detecting Outliers with DBSCAN

To detect outliers, using **DBSCAN** algorithm, that is a clustering method that finds groups of similar data points based on how close they are to each other. It works well for accelerometer and gyroscope data because it doesn't expect the data to follow a normal shape. Instead, it looks for dense areas in the data and marks anything far from these areas as an outlier. This makes it great for detecting outliers in sensor data, which often varies a lot.

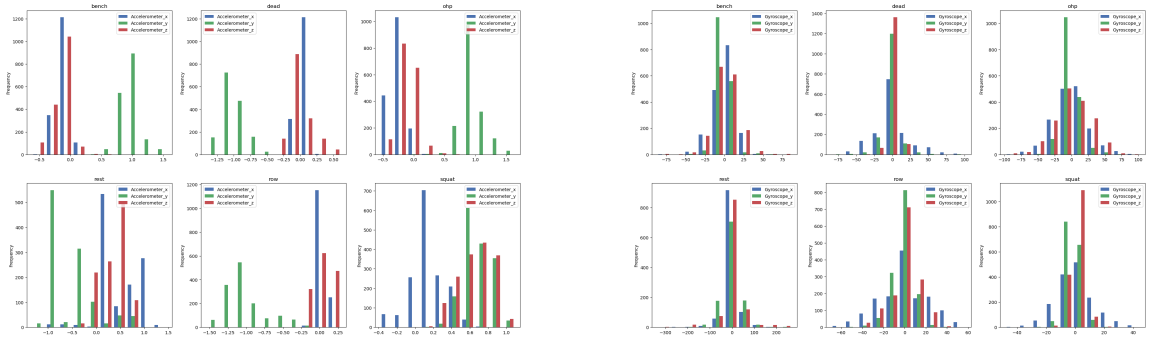

Figure 3: Normal Distribution in Gyroscope, Accelerometer.

As shown in the figure, our sensor data doesn't follow a normal (bell-shaped) distribution.

Dealing with Outliers

After detecting outliers using **DBSCAN**, we do not want to remove them to avoid introducing gaps in the time-series data. Instead, we used **linear interpolation** to estimate and replace the outlier values.

The process worked as follows:

- DBSCAN identified outliers and labeled them with `-1`.
- For each selected feature (e.g., accelerometer and gyroscope data), we retained the original values and applied interpolation only where outliers were detected.
- **Linear interpolation** estimates the missing or noisy values by drawing a straight line between the values immediately before and after the outlier, filling in the gap with a value along that line.

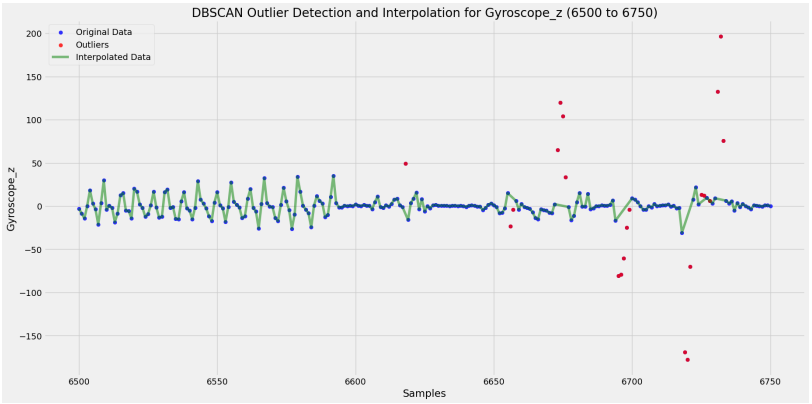

Figure 4: Corrected Signal Using Interpolation After Outlier Removal.

Feature Engineering

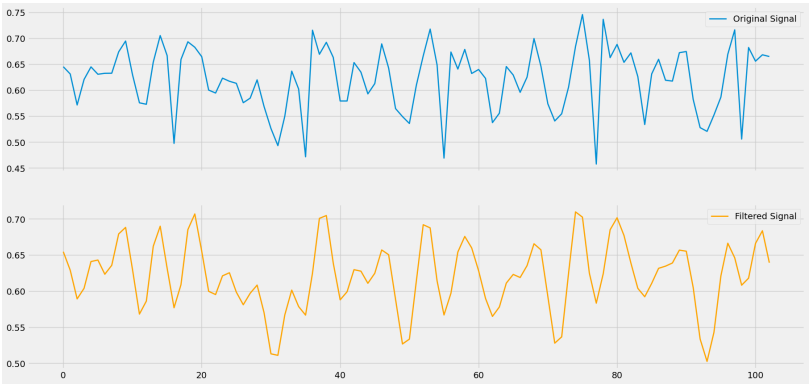

Figure 5: Transformation before and after the filter.

In the feature engineering step, is applied a low-pass filter to smooth the sensor data. The low-pass filter removes unwanted high-frequency noise, like sensor errors and vibrations, while keeping the important slower signals that show real body movement.

The cutoff frequency to 1.2 Hz to keep the slower movements from exercises, like squats, and remove faster noise. By removing the noise, the data becomes easier to analyze, leading to more accurate results.

Feature Extraction

1. Principal Component Analysis (PCA)

Before applying PCA, the data was normalized to ensure a fair comparison between larger and smaller values.

PCA was then applied to reduce the number of features for the gyroscope and accelerometer data. Using the elbow technique, the original six columns were reduced to three principal components: `pca_1`, `pca_2`, and `pca_3`.

These three PCA components were added to the dataset to see how well this technique perform in this case.

2. Magnitude Calculation

To measure the overall strength of motion, we compute the magnitude of the accelerometer and gyroscope signals using the Euclidean norm. This combines the x, y, and z components into a single value:

$$\text{Magnitude} = \sqrt{x^2 + y^2 + z^2}$$

This helps summarize 3D motion into one value for easier analysis, such as classifying different exercises.

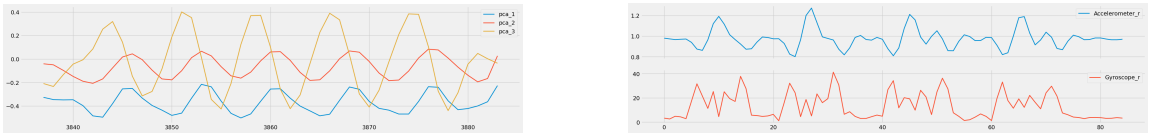

Figure 6: Extracted PCA components and squared magnitudes.

3. Abstract Mean

To make the sensor data smoother and easier to understand, a **moving average** is used. It takes a few values next to each other and finds their average, making the data smoother and less jumpy.

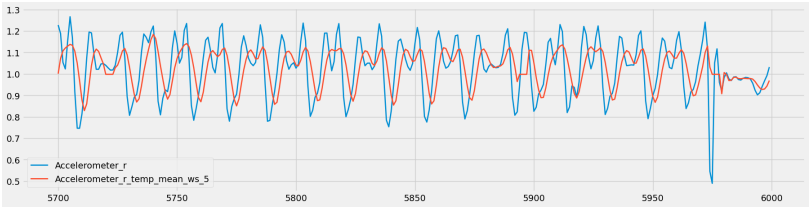

Figure 7: Extracting the abstract mean.

A **window size of 5** is used, meaning every 5 values are averaged together. This reduces noise and shows the overall trend in movement.

The moving average is applied to all sensor values, including the **accelerometer** and **gyroscope** magnitudes, for every data set.

NaN (missing) values can appear at the start, end, or even within the signal. At the start and end, there may be too few values to fill the window. Within the signal, NaNs occur if the original data contains missing values that affect the average calculation.

3.1 Handling NaN Values

To fix this, NaNs are replaced by the mean of each column, calculated while ignoring NaNs. This keeps the data smooth and complete without gaps.

Clustering

k-Means clustering is applied to both accelerometer and gyroscope data to identify patterns in motion signals. The number of clusters is chosen to match the number of exercise types, aiming to group similar movements together.

In Figure 8, each color represents a different cluster, showing how motion data points are separated based on their characteristics. These clusters are then used as labels for training the model to recognize specific types of exercises.

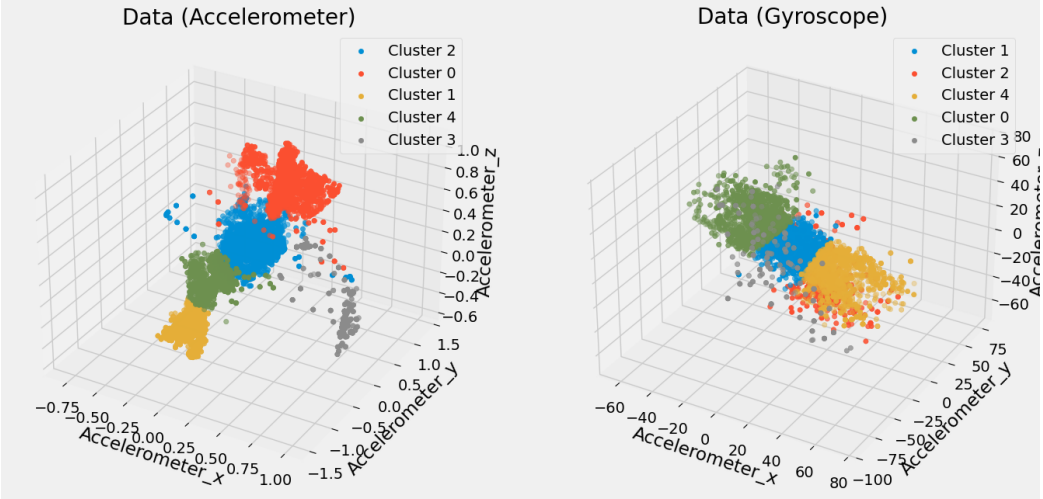

Figure 8: Clustering of accelerometer and gyroscope data.

Feature Correlation

Feature correlation was analyzed to identify variables that are strongly related to each other and to the target behavior.

The heatmap in Figure 9 shows the Pearson correlation coefficients between all extracted features. High correlation values (close to 1 or -1) indicate a strong linear relationship between two features, while values near 0 suggest weak or no correlation.

Two clear examples can be observed:

- `Accelerometer_x` and `Accelerometer_x_temp_mean_ws_5` have a very strong positive correlation ($r = 0.95$), since the temporal mean is directly computed from the original signal.
- `pca_1` shows high correlation with several original sensor features, indicating that the first principal component effectively captures a large portion of the data's variance.

Additionally, the clustering labels (`cluster_acc`, `cluster_gyro`) show moderate correlations with several features, confirming that the clustering process captures meaningful patterns in the data.

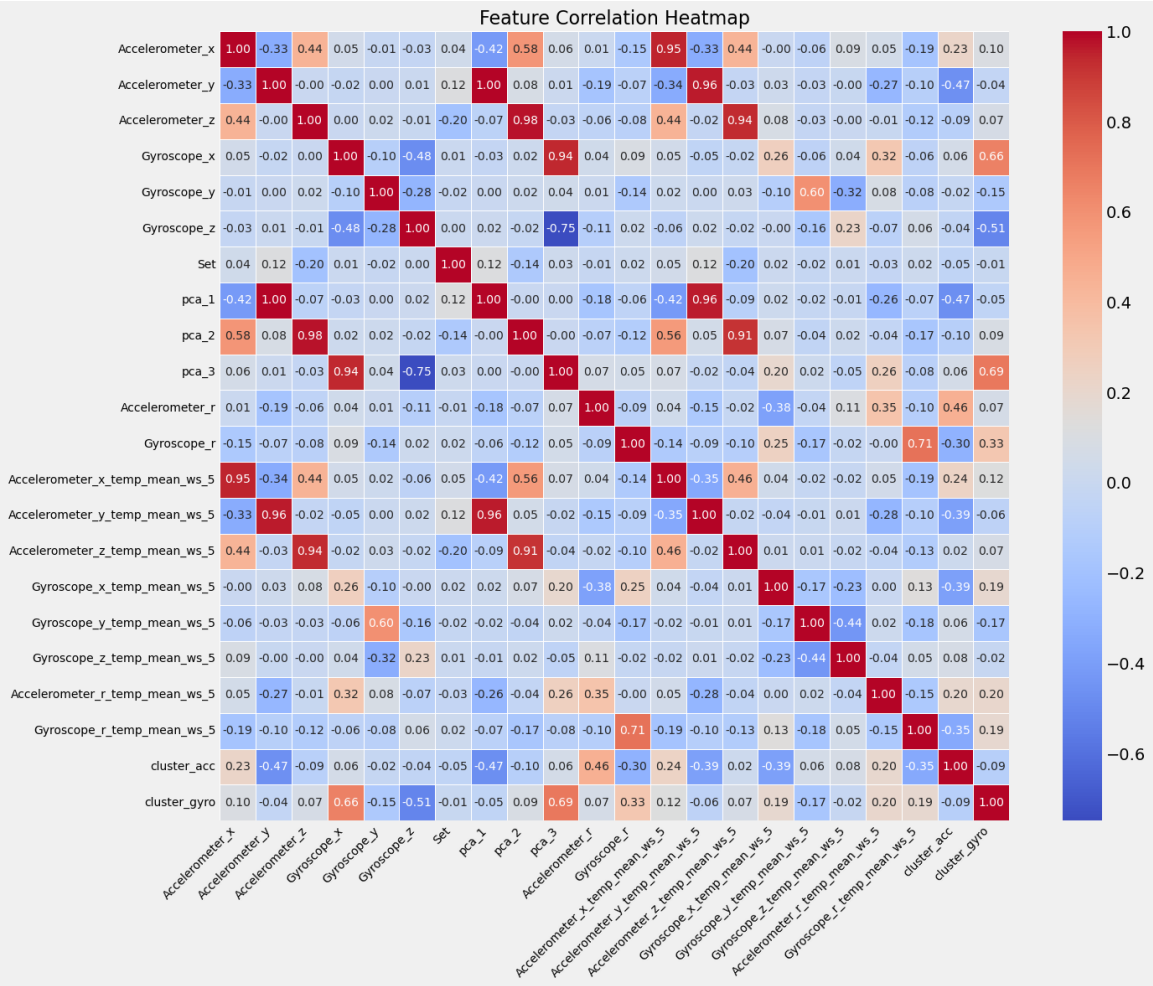

Figure 9: Pearson correlation heatmap.

Modeling Section

The number of samples for each activity was counted to understand how the data is distributed before training the model. The bar chart below shows how often each activity appears in the full dataset (*Total*), and how they are split between the training, validation and test sets.

The data was split using `train_test_split` with `stratify=y` to keep the same activity distribution in both sets. This helps the model learn from all activity types and gives a fair test.

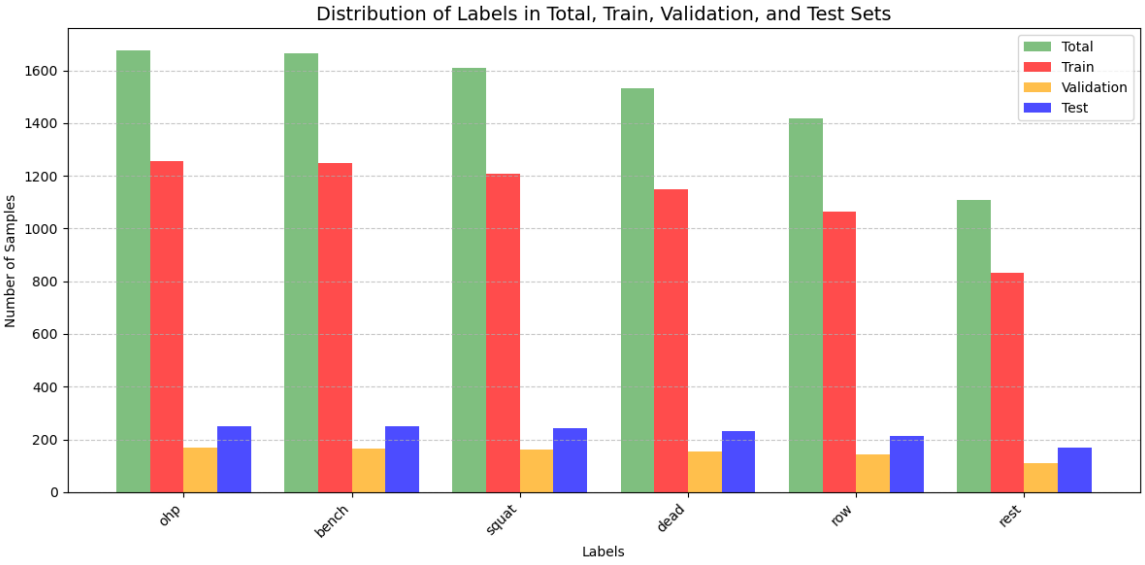

Figure 10: Distribution of activities in training and test sets.

Set Division

The features were grouped into five different sets:

- **Basic Features:** Raw accelerometer and gyroscope signals (x, y, z).
- **Square Features:** Signal magnitudes calculated from the raw sensor data.
- **PCA Features:** Features reduced using Principal Component Analysis (PCA).
- **Cluster Features:** Cluster labels from accelerometer and gyroscope data.

Mean features were initially excluded to observe how the model performs without them. Then, forward feature selection was used to identify the 10 most relevant features, which may include mean-based features.

These five sets are used to compare model performance across different feature combinations and understand the contribution of each type to classification accuracy.

Forward Selection using Decision Tree

A new set of features was selected using forward selection with a decision tree. This method adds one feature at a time, choosing the one that improves model accuracy the most at each step.

- Starts with no features.
- Each remaining feature is tested, and the one that helps the model the most is selected.

- The process repeats until the desired number of features is selected.
- The result is a list of features that improve performance step by step.

This helps identify the most important features for classification.
The final set of selected features is:

```
['pca_1', 'Accelerometer_z', 'Accelerometer_x', 'Gyroscope_r_temp_mean_ws_5',
'Accelerometer_y_temp_mean_ws_5', 'Gyroscope_r', 'Gyroscope_y',
'Accelerometer_x_temp_mean_ws_5', 'Accelerometer_r_temp_mean_ws_5', 'Gyroscope_z']
```

This list includes both original sensor features and mean-based features, showing that mean values can help improve model accuracy.

Models Performance & Algorithms

DBSCAN: Outlier Detection and Parameter Tuning

DBSCAN was used to detect outliers in the dataset. To improve its performance, the key parameters were carefully studied.

Main Parameters of DBSCAN:

- **eps** (epsilon): The maximum distance between two points to be considered neighbors.
- **min_samples**: The minimum number of neighboring points required to form a dense region (a cluster).

Silhouette Score: This score measures the quality of clustering, ranging from -1 to 1.
A score closer to 1 means the points are well grouped; a score near -1 suggests poor clustering.

Parameter Tuning Process:

- The dataset is first standardized so that all features have similar scales.
- DBSCAN is applied with different combinations of `eps` (0.1 to 2.0) and `min_samples` (5 to 20).
- For each combination, the silhouette score is calculated to assess clustering quality.
- A good combination gives a high silhouette score and less than 20% of the data marked as outliers. This ensures effective clustering while minimizing data loss.

Best Parameters Found:

$$\text{eps} = 1.6, \quad \text{min\_samples} = 7$$

This combination achieved the highest silhouette score of 0.4262, indicating well-formed clusters with minimal outliers.

K-Nearest Neighbor (KNN)

KNN classifies data points by looking at the majority class of their closest neighbors.

KNN Tuning

`GridSearchCV` was used to find the best number of neighbors for each dataset. The goal was to avoid overfitting, so the selection was not only based on accuracy, but also on how well the model performed on both the training and validation sets.

As shown in the figure, KNN performs better on the selected features because these features give clearer and more useful information. They help the model find similar patterns more easily, which improves classification.

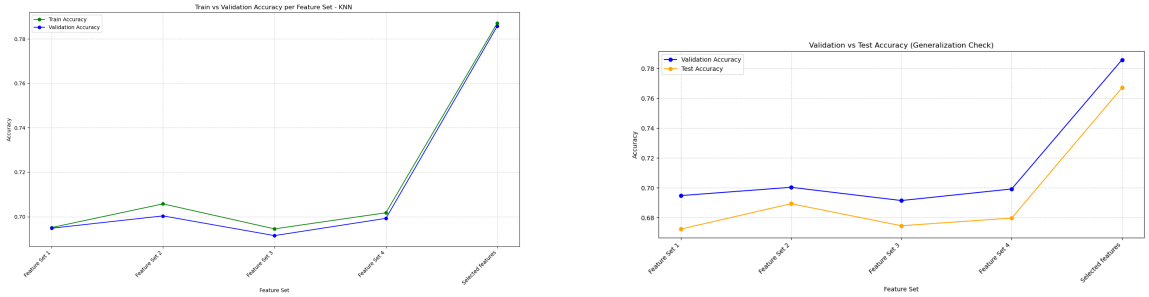

Figure 11: Knn Tuning Performace.

KNN Performance

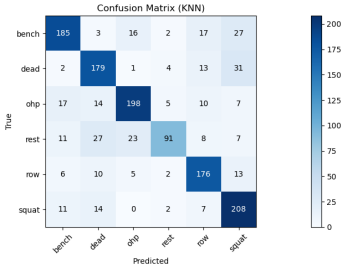

Figure 12: Best KNN performance on Validation.

KNN performed well on the selected features, with an accuracy of 0.7670, which is a good result. However, as shown in the confusion matrix, KNN can be affected by noise because it also considers noisy points when making predictions.

Naive Bayes (NB)

Naive Bayes is a simple model that assumes all features are independent. No parameter tuning was applied, as it typically performs well with its default settings.

As we see in the figure, it performed better on the selected features because these features are more informative. This helps Naive Bayes make better predictions, as it works well when features are independent and clearly separate the classes.

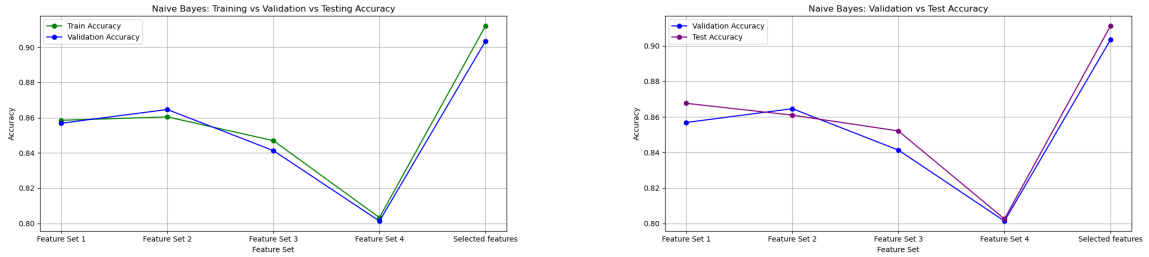

Figure 13: NB Tuning Performace.

NB Performance

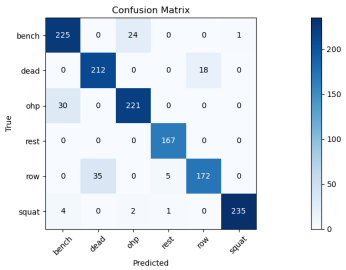

Figure 14: Confusion matrix for NB.

Naive Bayes performed better on the selected features, achieving an accuracy of 0.9112. It handles noise more effectively, but sometimes gets confused when exercises are very similar, as it may not fully capture the differences between them.

Logistic Regression (LR)

Logistic Regression is a simple linear model that predicts class probabilities based on input features. It works by finding the best line that separates the classes.

LR Tuning

`GridSearchCV` was used to tune the hyperparameters of the model for each dataset. The tuning was based on validation accuracy to find the best combination that balances performance and generalization.

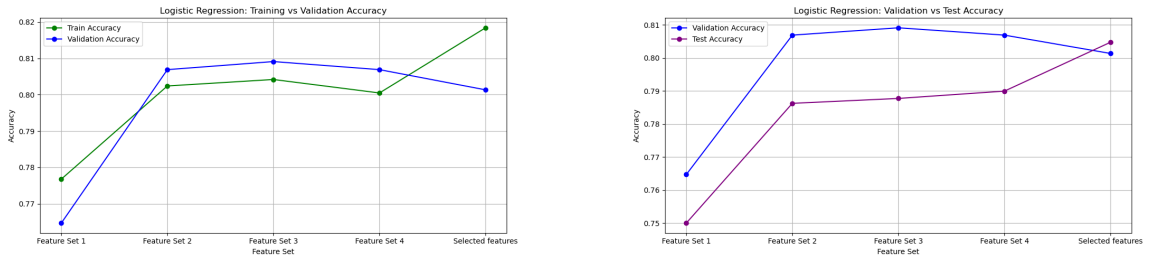

Figure 15: Logistic Regression Tuning Performance.

Logistic Regression's performance improves across the feature sets, reaching the highest accuracy of 0.8047 with the selected features. This means the selected features are more informative, helping the model find clearer patterns and predict better.

Sometimes, validation accuracy is higher than training accuracy. This can happen by chance or if the validation data is easier to predict. Overfitting usually shows the opposite pattern, with training accuracy higher than validation accuracy. Overall, these results show that good feature selection helps reduce the risk of overfitting.

LR Performance

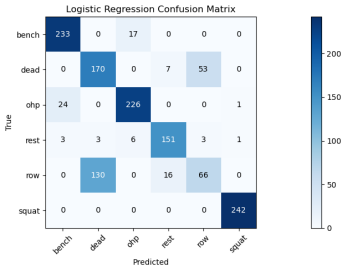

Figure 16: Cofusion matrix for LR.

Logistic Regression achieved an accuracy of 0.8047 on the selected features. It handled the data fairly well, thanks to regularization, but performed slightly worse than Naive Bayes because it assumes a linear relationship. This can be too simple for complex and noisy sensor patterns.

Support Vector Machine (SVM) without Kernel

Linear SVM tries to find the best straight line that separates the classes by maximizing the margin between them. It works well by balancing fitting the data and keeping the decision boundary simple.

SVM Tuning

`GridSearchCV` was used to tune hyperparameters. The tuning was based on validation accuracy to find the best balance and avoid overfitting.

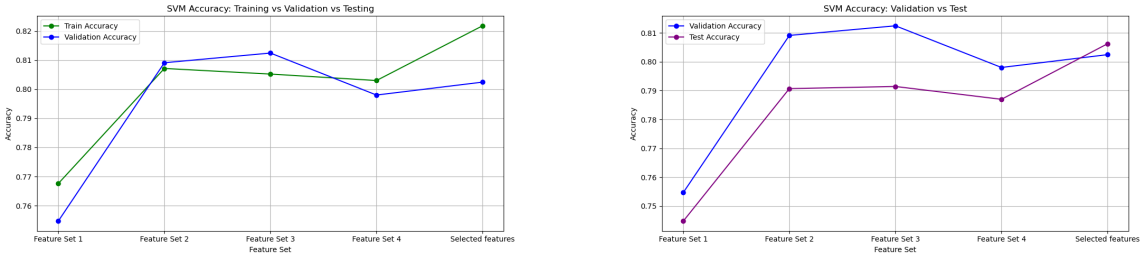

Figure 17: SVM Tuning Performance.

SVM's accuracy improves as the feature sets become better and more informative, showing that it benefits from clearer features.

However, in some cases, validation accuracy is slightly higher than training accuracy. This can happen due to random variation or if the validation data is easier, especially when working with

complex features.

Overall, these results confirm that better feature selection helps the SVM perform more accurately while reducing the risk of overfitting.

SVM Performance

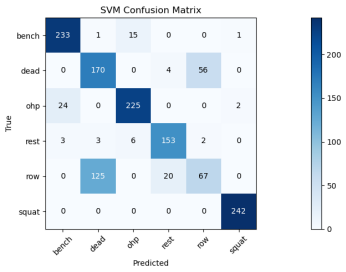

Figure 18: Confusion matrix for SVM without kernel.

The best test accuracy of 0.8062 was achieved using the selected features. SVM achieved good accuracy and performed well, but it separates classes using a straight line, which can be too simple for noisy sensor data with complex patterns.

Decision Tree (DT)

Decision Trees create a model by splitting the data based on feature values, forming a tree structure that makes decisions step-by-step.

DT Tuning

`GridSearchCV` was used to tune the hyperparameters of the decision tree, including larger values for $min\_samples\_leaf$ to help prevent overfitting. The tuning focused on validation accuracy to find the best balance between performance and generalization.

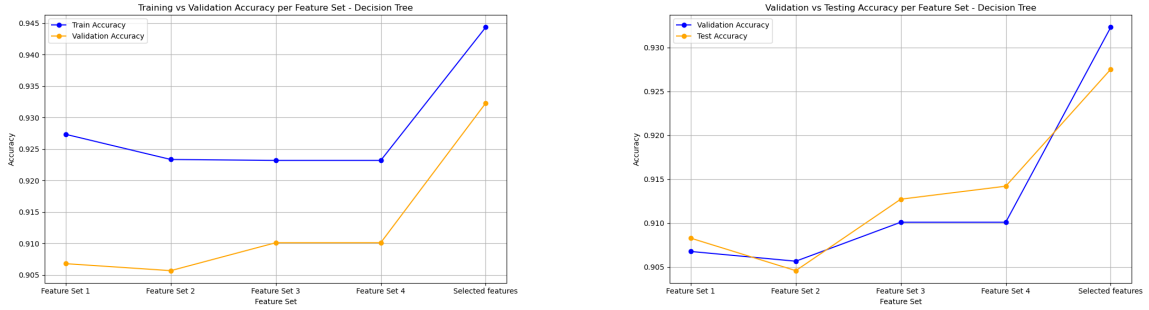

Figure 19: Decision Tree Tuning Performance.

Decision Tree accuracy improved with better feature sets, showing it benefits from clearer, more informative features. The results show that good feature selection helps Decision Trees perform better and generalize well.

In some cases, test accuracy is slightly higher than validation accuracy. This may be due to random variation or because the model was tuned using the validation set, allowing it to generalize

better to the unseen test data.

DT Performance

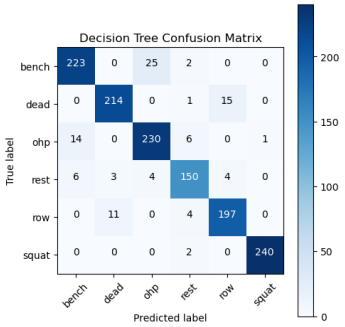

Figure 20: Confusion matrix for Decision Tree.

The best test accuracy of 0.9275 was achieved using the selected features. Decision Tree performed very well because it can handle complex data by making step-by-step decisions based on feature values.

Random Forest (RF)

Random Forest creates a model by building many decision trees and combining their results. This improves accuracy and helps reduce overfitting.

RF Tuning

`GridSearchCV` was used to find the best hyperparameters, focusing on validation accuracy to balance performance and generalization.

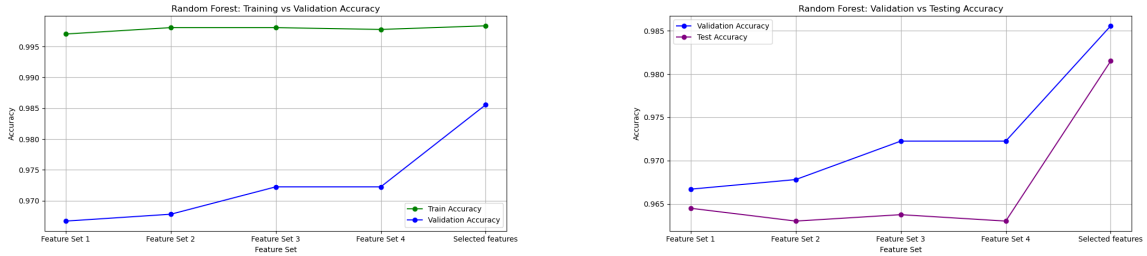

Figure 21: Random Forest Tuning Performance.

Random Forest accuracy got better with improved features, showing it works well with clear, useful data. Validation and test accuracy are similar, meaning it generalizes well.

RF Performance

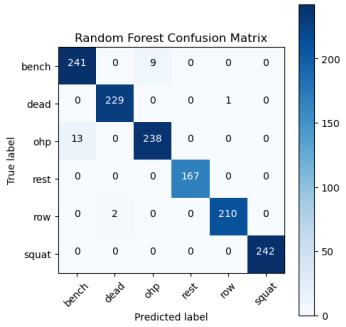

Figure 22: Confusion matrix for Random Forest.

The best test accuracy came from using the selected features with accuracy: 0.9845. Random Forest did better than the others because it uses many trees that look at the data in different ways. By combining these trees, it reduces mistakes from noisy data and finds hidden patterns more easily. This makes it work very well with complex sensor data.

Models Comparison

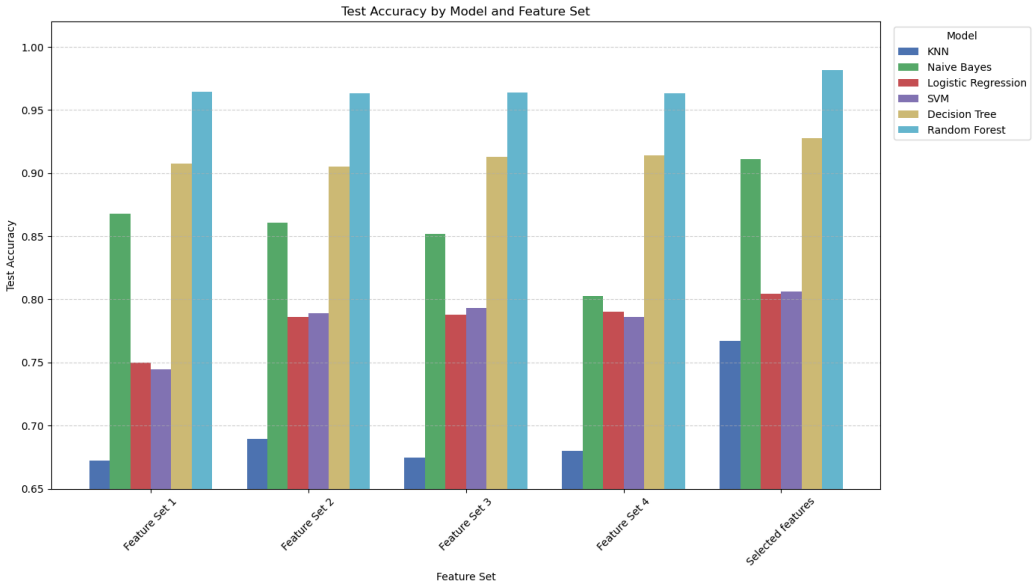

Figure 23: Models Results.

Random Forest is the best model, reaching a test accuracy of **0.9815** with the selected features. It works well because it combines many decision trees, which helps reduce overfitting and capture complex patterns in the sensor data.

Decision Tree also performed well (**0.9275**), but relying on a single tree can lead to less stable and less generalizable results.

SVM and Logistic Regression gave good results (around **0.80**) and handled the data fairly well.

Naive Bayes did better than expected (**0.9112**) because it works well with simple feature relationships.

KNN had the lowest accuracy but improved with better features (**0.7670**). It struggles more with noisy or high-dimensional data.

	Precision						Recall						F1-Score					
Model	Decision Tree	Knn	Logistic Regression	Naive Bayes	Random Forest	SVM	Decision Tree	Knn	Logistic Regression	Naive Bayes	Random Forest	SVM	Decision Tree	Knn	Logistic Regression	Naive Bayes	Random Forest	SVM
Class																		
bench	0.918	0.797	0.896	0.869	0.949	0.896	0.892	0.740	0.932	0.900	0.964	0.932	0.905	0.768	0.914	0.884	0.956	0.914
dead	0.939	0.725	0.561	0.858	0.991	0.569	0.930	0.778	0.739	0.922	0.996	0.739	0.934	0.751	0.638	0.889	0.993	0.643
ohp	0.888	0.815	0.908	0.895	0.964	0.915	0.916	0.789	0.900	0.880	0.948	0.896	0.902	0.802	0.904	0.888	0.956	0.905
rest	0.909	0.858	0.868	0.965	1.000	0.864	0.898	0.545	0.904	1.000	1.000	0.916	0.904	0.667	0.886	0.982	1.000	0.890
row	0.912	0.762	0.541	0.905	0.995	0.536	0.929	0.830	0.311	0.811	0.991	0.316	0.921	0.795	0.395	0.856	0.993	0.398
squat	0.996	0.710	0.992	0.996	1.000	0.988	0.992	0.860	1.000	0.971	1.000	1.000	0.994	0.778	0.996	0.983	1.000	0.994

Figure 24: Scores.

The table shows how different models performed on exercises using accelerometer and gyroscope data. Precision, recall, and F1-score are stable, indicating no overfitting.

Random Forest had the highest precision overall, making fewer false positives and being the most accurate, especially on exercises like *squat* and *deadlift*.

KNN showed decent recall but generally lower precision, struggling more with exercises that have similar sensor patterns like *row* and *deadlift*.

The **SVM without kernel** and **Logistic Regression** also had difficulty distinguishing these similar exercises, resulting in lower scores.

Other models had mixed results but usually performed worse than Random Forest.

Conclusion

This project focused on analyzing raw accelerometer and gyroscope data by extracting important features like mean and magnitude and applying filters to clean the signals. Several machine learning models were tested to classify different gym exercises. Among them, Random Forest performed best due to its ability to handle complex and noisy data. The project showed that careful data preparation and feature extraction play a crucial role in improving model accuracy. Overall, the work demonstrates the importance of each step in the machine learning process when working with sensor data, from understanding the signals to selecting and tuning models.

Bibliography

- Shoaib, M., Bosch, S., Incel, O. D., Scholten, H., & Havinga, P. J. M. (2021). Complex Human Activity Recognition Using Smartphone and Wrist-Worn Motion Sensors. *IEEE Sensors Journal*, 21(12), 13352–13363. `https://ieeexplore.ieee.org/document/9430501`
- Mukherjee, S., & Sharma, N. (2021). Machine Learning Approaches for Human Activity Recognition: A Comparative Study. *arXiv preprint*. `https://arxiv.org/abs/2109.01118`
- Wikipedia. (n.d.). Gyroscope. `https://en.wikipedia.org/wiki/Gyroscope`
- Wikipedia. (n.d.). Fitness tracker. `https://en.wikipedia.org/wiki/Fitness_tracker`